

C# Programs:

C# Projects: 4th week

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.IO;
using System.Threading.Tasks;
```

class Program

```
{
    static List<Book> books=new List<Book>();

    // dict for fast lookup
    static Dictionary<int, Book> bookDict=new Dictionary<int, Book>();

    // books properties using array
    static Book[] bookArray =
{ // like Book b1=new Book()->b1.name="",id="" new book is object for Book array
    new Book { Id = 101, Title = "C# Programming", Author = "John Smith", IsAvailable = true },
    new Book { Id = 102, Title = "Java Basics", Author = "James Gosling", IsAvailable = true },
    new Book { Id = 103, Title = "Python for Beginners", Author = "Guido van Rossum", IsAvailable = false },
    new Book { Id = 104, Title = "Data Structures", Author = "Mark Allen", IsAvailable = true }
};
```

static async Task Main()

```
{

// Load books from file at startup
if (File.Exists("books.txt"))
{
    string[] lines = File.ReadAllLines("books.txt");

    foreach (string line in lines)
    {
        string[] data = line.Split(',');

        Book b = new Book
        {
            Id = int.Parse(data[0]),
            Title = data[1],
            Author = data[2],
            IsAvailable = bool.Parse(data[3])
        };

        books.Add(b); // adding loaded books to list
        Console.WriteLine("Loaded Book: " + b.Title + " Id: " + b.Id);
    }
}
else
{
    AddBook(); // if file not present load from array
}
```

ViewBooks();

```
try
{
```

```

        Console.WriteLine("Removing book with Id 102...");
        RemoveBook(102);
    }
    catch (BookNotFoundException ex)
    {
        Console.WriteLine(ex.Message);
    }
    finally
    {
        Console.WriteLine("Remove operation finished.");
    }

    Console.WriteLine("\nUpdated Book List:");
    ViewBooks();

    await SaveBooksAsync();

    //adding books to dict from array
    foreach(Book b in bookArray)
    {
        bookDict.Add(b.Id, b);
    }

    //Borrow and Return Books (Queue) queue for waiting users
    int bookId = 101; //for this book waitng users queue is
    Queue<string> waitingUsers=new Queue<string>();
    waitingUsers.Enqueue("divya");
    waitingUsers.Enqueue("sony");

    Console.WriteLine("Next user: " + waitingUsers.Peek());

    string user = waitingUsers.Dequeue();
    Console.WriteLine(user + " borrowed the book with Id:" + bookId);

    //UNDO last book removal
    Stack<Book> removedBooks = new Stack<Book>();
    Book bookToRemove = books[0]; // take first book to remove
    books.Remove(bookToRemove); // remove from list
    removedBooks.Push(bookToRemove); // store the removed from list to stack for undo

    if (removedBooks.Count > 0)//stack has atleast one elemnet
    {
        Book lastRemovedBook = removedBooks.Pop(); // get last removed book
        books.Insert(0,lastRemovedBook);
        Console.WriteLine("Undo the deleted book:");
        Console.WriteLine("Id: " + lastRemovedBook.Id);
    }

    //UNDO last borrow
    Stack<string> borrowHistory = new Stack<string>();
    borrowHistory.Push(user); //takes user from dequeue
    if (borrowHistory.Count > 0)
    {
        string lastUser = borrowHistory.Pop();
        waitingUsers.Enqueue(lastUser);
        Console.WriteLine("Undo borrow: " + lastUser);
    }

```

```

Console.WriteLine("books with John Smith");
// find book by author
var result = books.Where(b => b.Author == "John Smith");

foreach (var book in result)
{
    Console.WriteLine(book.Title);
}

Console.WriteLine("available Books");
// find available books
var availableBooks = books.Where(b => b.IsAvailable);

foreach (var book in availableBooks)
{
    Console.WriteLine(book.Title);
}

}

//add Book
static void AddBook()
{
    foreach(var b in bookArray)
    {
        books.Add(b);
    }
}

// to remove damaged , unwanted books
static void RemoveBook(int id)//we are selecting book to remove that id is this
{
    Book ? bookToRemove=null;

    foreach(Book b in books)
    {
        if (b.Id == id) //of remove book matched to list book
        {
            bookToRemove=b;//that book
            break;
        }
    }

    if (bookToRemove == null)
    {
        throw new BookNotFoundException("Book not found with Id: " + id);
    }

    books.Remove(bookToRemove);
    Console.WriteLine("Book removed successfully");
}

//to display all books
static void ViewBooks()
{
    if (books.Count == 0)
    {
        Console.WriteLine("No books available.");
    }
}

```

```

        return;
    }

    foreach(Book b in books)
    {
        Console.WriteLine("Id: " + b.Id);
        Console.WriteLine("Title: " + b.Title);
        Console.WriteLine("Author: " + b.Author);
        Console.WriteLine("Available: " + b.IsAvailable);
        Console.WriteLine("-----");
    }
}

//Async Operation
//Simulate database saving.
static async Task SaveBooksAsync()
{
    Console.WriteLine("Saving books to database...");

    await Task.Delay(2000); // simulate database delay (2 seconds)

    await File.WriteAllLinesAsync("books.txt",
        books.Select(b => b.Id + "," + b.Title + "," + b.Author + "," + b.IsAvailable));

    Console.WriteLine("Books saved successfully.");
}

}

class Book
{
    public int Id { get; set; }
    public string Title { get; set; }
    public string Author { get; set; }
    public bool IsAvailable { get; set; }
}

//Create custom exception:
//BookNotFoundException
class BookNotFoundException : Exception
{
    public BookNotFoundException(string message) : base(message)
    {
    }
}

*****

Project2:
using System;
using System.Collections.Generic;
using System.Linq;
using System.IO;
using System.Text.Json;
using System.Threading.Tasks;

class TaskItem
{
    public int TaskId { get; set; }
    public string Title { get; set; }
}

```

```

    public string StudentName { get; set; }
    public DateTime Deadline { get; set; }
    public bool IsCompleted { get; set; }
}

// Custom Exception
class InvalidTaskException : Exception
{
    public InvalidTaskException(string message) : base(message)
    {
    }
}

class Program
{
    static List<TaskItem> tasks = new List<TaskItem>();
    static Dictionary<int, TaskItem> taskLookup = new Dictionary<int, TaskItem>();
    static HashSet<string> students = new HashSet<string>();
    static Queue<TaskItem> taskQueue = new Queue<TaskItem>();

    static string filePath = "tasks.json";

    static async Task Main(string[] args)
    {
        LoadTasks();

        while (true)
        {
            Console.WriteLine("\n--- Student Task Tracker ---");
            Console.WriteLine("1 Add Task");
            Console.WriteLine("2 Complete Task");
            Console.WriteLine("3 View Tasks");
            Console.WriteLine("4 Overdue Tasks");
            Console.WriteLine("5 Exit");

            Console.Write("Choose option: ");
            int choice = int.Parse(Console.ReadLine());

            try
            {
                switch (choice)
                {
                    {
                        case 1:
                            AddTask();
                            break;

                        case 2:
                            Console.Write("Enter Task Id: ");
                            int id = int.Parse(Console.ReadLine());

                            await Task.Run(() => CompleteTask(id));
                            break;

                        case 3:
                            ViewTasks();
                            break;

                        case 4:

```

```

        ViewOverdueTasks();
        break;

    case 5:
        await SaveTasksAsync();
        return;

    default:
        Console.WriteLine("Invalid choice");
        break;
    }
}
catch (InvalidTaskException ex)
{
    Console.WriteLine(ex.Message);
}
catch (FormatException)
{
    Console.WriteLine("Invalid input");
}
}
}

static void AddTask()
{
    TaskItem task = new TaskItem();

    Console.Write("Task Id: ");
    task.TaskId = int.Parse(Console.ReadLine());

    Console.Write("Title: ");
    task.Title = Console.ReadLine();

    Console.Write("Student Name: ");
    task.StudentName = Console.ReadLine();

    Console.Write("Deadline (yyyy-mm-dd): ");
    task.Deadline = DateTime.Parse(Console.ReadLine());

    task.IsCompleted = false;

    tasks.Add(task);
    taskLookup[task.TaskId] = task;
    students.Add(task.StudentName);
    taskQueue.Enqueue(task);

    Console.WriteLine("Task added successfully");
}

static void CompleteTask(int id)
{
    if (!taskLookup.ContainsKey(id))
    {
        throw new InvalidTaskException("Task not found with Id: " + id);
    }

    taskLookup[id].IsCompleted = true;
}

```

```

        Console.WriteLine("Task completed by student: " + taskLookup[id].StudentName);
    }

    static void ViewTasks()
    {
        foreach (var t in tasks)
        {
            Console.WriteLine($"Id:{t.TaskId} Title:{t.Title} Student:{t.StudentName} Deadline:{t.Deadline.ToShortDateString()}
Completed:{t.IsCompleted}");
        }
    }

    static void ViewOverdueTasks()
    {
        var overdue = tasks.Where(t => t.Deadline < DateTime.Now && !t.IsCompleted);

        Console.WriteLine("\nOverdue Tasks:");

        foreach (var t in overdue)
        {
            Console.WriteLine($"{t.Title} - {t.StudentName}");
        }
    }

    static void LoadTasks()
    {
        if (File.Exists(filePath))
        {
            string json = File.ReadAllText(filePath);

            tasks = JsonSerializer.Deserialize<List<TaskItem>>(json);

            foreach (var task in tasks)
            {
                taskLookup[task.TaskId] = task;
                students.Add(task.StudentName);
                taskQueue.Enqueue(task);
            }

            Console.WriteLine("Tasks loaded from file.");
        }
    }

    static async Task SaveTasksAsync()
    {
        string json = JsonSerializer.Serialize(tasks, new JsonSerializerOptions
        {
            WriteIndented = true
        });

        await File.WriteAllTextAsync(filePath, json);

        Console.WriteLine("Tasks saved to file.");
    }
}

```

JavaScript

Problem1:

// Fake API 1

```
function getUsers() {
  return new Promise(resolve => {
    setTimeout(() => {
      resolve([
        { id: 1, name: "Rahul" },
        { id: 2, name: "Anita" }
      ]);
    }, 1000);
  });
}
```

// Fake API 2

```
function getOrders() {
  return new Promise(resolve => {
    setTimeout(() => {
      resolve([
        { userId: 1, amount: 500 },
        { userId: 1, amount: 800 },
        { userId: 2, amount: 300 }
      ]);
    }, 1200);
  });
}
```

// Fake API 3

```
function getPayments() {
  return new Promise(resolve => {
    setTimeout(() => {
      resolve([]);
    }, 1500);
  });
}
```

// Main program

```
async function main() {
```

// 1. Fetch all APIs in parallel

```
const [users, orders, payments] = await Promise.all([
  getUsers(),
  getOrders(),
  getPayments()
]);
```

// 2. Calculate total order amount per user

```
const results = users.map(user => {
  const total = orders
    .filter(order => order.userId === user.id)
    .reduce((sum, order) => sum + order.amount, 0);
```

```
  return {
    name: user.name,
    total
  };
});
```

// 3. Print results


```

    results.forEach(r => {
      console.log(`${r.name} → ${r.total}`);
    });
  }

  main();
  *****

  problem2:
  const transactions = [
    { category: "Food", amount: 200 },
    { category: "Travel", amount: 500 },
    { category: "Food", amount: 300 },
    { category: "Shopping", amount: 800 },
    { category: "Travel", amount: 200 }
  ];

  // 1. Group and sum by category using reduce
  const grouped = transactions.reduce((acc, t) => {
    if (!acc[t.category]) {
      acc[t.category] = 0;
    }
    acc[t.category] += t.amount;
    return acc;
  }, {});

  // 2. Convert object to array
  const result = Object.entries(grouped);

  // 3. Sort highest to lowest
  result.sort((a, b) => b[1] - a[1]);

  // Print result
  result.forEach(([category, total]) => {
    console.log(`${category} → ${total}`);
  });
  *****

  Problem3:
  const tasks = [1,2,3,4,5,6,7,8,9,10]; // task queue
  let index = 0;

  // simulate task with random time
  function runTask(id){
    return new Promise(resolve => {

      const time = Math.floor(Math.random()*2000) + 1000;

      console.log("Starting Task", id);

      setTimeout(()=>{
        console.log("Completed Task", id);
        resolve();
      }, time);

    });
  }

  // worker function

```

```

async function worker(){

  while(index < tasks.length){

    const taskId = tasks[index];
    index++;

    await runTask(taskId);

  }

}

// main processor
async function processTasks(){

  const w1 = worker(); //concurrency limit(3 workers does 3 tasks that a time
  const w2 = worker();
  const w3 = worker(); //fixed worker can do no of tasks thread pooling

  await Promise.all([w1, w2, w3]); //wait until task start,complete , promise resolve

}

```

```

processTasks();
*****

```

problem4:

```

<!DOCTYPE html>
<html>
<head>
<title>Employee Filter</title>
</head>

<body>

<h2>Employee Table</h2>

<select id="departmentFilter">
  <option value="All">All</option>
  <option value="IT">IT</option>
  <option value="HR">HR</option>
  <option value="Finance">Finance</option>
</select>

```

```

<table border="1">
  <thead>
    <tr>
      <th>Name</th>
      <th>Department</th>
      <th>Salary</th>
    </tr>
  </thead>

  <tbody id="tableBody"></tbody>

```

```

</table>

```

```

<h3 id="avgSalary"></h3>

```

```

<script>

const employees = [
  {name:"Rahul", department:"IT", salary:60000},
  {name:"Anita", department:"HR", salary:50000},
  {name:"Karan", department:"IT", salary:70000},
  {name:"Priya", department:"Finance", salary:65000}
];

const tableBody = document.getElementById("tableBody");
const filtery = document.getElementById("departmentFilter");
const avgText = document.getElementById("avgSalary");

function displayEmployees(data){

  tableBody.innerHTML = "";

  data.map(emp => {

    const row = `<tr>
      <td>${emp.name}</td>
      <td>${emp.department}</td>
      <td>${emp.salary}</td>
    </tr>`;

    tableBody.innerHTML += row;

  });

  const total = data.reduce((sum, emp) => sum + emp.salary, 0);

  const avg = data.length ? total / data.length : 0;

  avgText.textContent = "Average Salary: " + avg;

}

filtery.addEventListener("change", function(){

  const dept = filtery.value;

  const filtered =
    dept === "All"
      ? employees
      : employees.filter(emp => emp.department === dept);

  displayEmployees(filtered);

});

displayEmployees(employees);

</script>

</body>
</html>

```

```
problem5: <!DOCTYPE html>
<html>
<head>
<title>Search Suggestion</title>
<style>
.highlight{
  color:red;
  font-weight:bold;
}
</style>
</head>

<body>

<h2>Search Products</h2>

<input type="text" id="searchBox" placeholder="Type to search...">

<ul id="suggestions"></ul>

<script>

const products = [
  "laptop",
  "lamp",
  "laser printer",
  "keyboard",
  "mouse",
  "monitor",
  "mobile"
];

const searchBox = document.getElementById("searchBox");
const suggestionList = document.getElementById("suggestions");

searchBox.addEventListener("input", function(){

  const text = searchBox.value.toLowerCase();

  suggestionList.innerHTML = "";

  const filtered = products.filter(p => p.toLowerCase().startsWith(text));

  filtered.forEach(item => {

    const li = document.createElement("li");

    const highlight = item.substring(0, text.length);

    const rest = item.substring(text.length);

    li.innerHTML = `<span class="highlight">${highlight}</span>${rest}`;

    suggestionList.appendChild(li);

  });

});
```

```
});
```

```
</script>
```

```
</body>
```

```
</html>
```

Problem6:

```
const arr = [
```

```
  [1,2,3],
```

```
  [4,5],
```

```
  [6,7,8,9]
```

```
];
```

```
// 1. Flatten the array
```

```
const flatArray = arr.flat();
```

```
// 2. Remove duplicates
```

```
const unique = [...new Set(flatArray)];
```

```
// 3. Keep only even numbers
```

```
const evens = unique.filter(num => num % 2 === 0);
```

```
// 4. Sort descending
```

```
const result = evens.sort((a,b) => b - a);
```

```
console.log(result);
```

```
*****
```

```
1* htmlfile
```

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<title>Multi Step Form</title>
```

```
<style>
```

```
.step{display:none}
```

```
.step.active{display:block}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<h2>Profile Form</h2>
```

```
<form id="form">
```

```
<!-- Step 1 -->
```

```
<div class="step" id="step1">
```

```
<h3>Step 1 – Personal Info</h3>
```

```
<label>Name</label>
```

```
<input type="text" id="name"><br><br>
```

```
<label>Email</label>
```

```
<input type="email" id="email"><br><br>
```

```
<label>Age</label>
```

```
<input type="number" id="age"><br><br>
```

```

<button type="button" id="next1">Next</button>
</div>

<!-- Step 2 -->
<div class="step" id="step2">
<h3>Step 2 – Skills</h3>

<input list="skillsList" id="skillInput">
<datalist id="skillsList">
<option value="JavaScript">
<option value="Python">
<option value="Java">
<option value="Go">
</datalist>

<button type="button" id="addSkill">Add Skill</button>

<ul id="skills"></ul>

<br>

<button type="button" id="prev1">Previous</button>
<button type="button" id="next2">Next</button>

</div>

<!-- Step 3 -->
<div class="step" id="step3">
<h3>Step 3 – Profile Strength</h3>

<progress id="progressBar" value="0" max="100"></progress>
<span id="percent"></span>

<br><br>

<button type="button" id="prev2">Previous</button>

</div>

</form>

<script src="script.js"></script>

</body>
</html>
JS File:
let currentStep = 1

const steps = {
1: document.getElementById("step1"),
2: document.getElementById("step2"),
3: document.getElementById("step3")
}

const nameInput = document.getElementById("name")
const emailInput = document.getElementById("email")
const ageInput = document.getElementById("age")

```

```

const skillInput = document.getElementById("skillInput")
const skillsList = document.getElementById("skills")

const progressBar = document.getElementById("progressBar")
const percent = document.getElementById("percent")

let skills = []

function showStep(step){
  Object.values(steps).forEach(s=>s.classList.remove("active"))
  steps[step].classList.add("active")
  currentStep = step
  localStorage.setItem("step", step)
  updateProgress()
}

function saveData(){
  const data = {
    name: nameInput.value,
    email: emailInput.value,
    age: ageInput.value,
    skills: skills
  }
  localStorage.setItem("formData", JSON.stringify(data))
  updateProgress()
}

function loadData(){

  const data = JSON.parse(localStorage.getItem("formData"))

  if(data){
    nameInput.value = data.name || ""
    emailInput.value = data.email || ""
    ageInput.value = data.age || ""
    skills = data.skills || []
    renderSkills()
  }

  const savedStep = localStorage.getItem("step")
  if(savedStep){
    showStep(parseInt(savedStep))
  }else{
    showStep(1)
  }

  updateProgress()
}

function renderSkills(){
  skillsList.innerHTML = ""
  skills.forEach(skill=>{
    const li = document.createElement("li")
    li.textContent = skill
    skillsList.appendChild(li)
  })
}

```

```
document.getElementById("addSkill").onclick = () =>{  
const skill = skillInput.value.trim()
```

```
if(skill){  
skills.push(skill)  
skillInput.value=""  
renderSkills()  
saveData()  
}  
}
```

```
function validateStep1(){
```

```
if(!nameInput.value.trim()){  
alert("Enter name")  
return false  
}
```

```
if(!emailInput.value.trim()){  
alert("Enter email")  
return false  
}
```

```
if(!ageInput.value.trim()){  
alert("Enter age")  
return false  
}
```

```
return true  
}
```

```
document.getElementById("next1").onclick = ()=>{
```

```
if(validateStep1()){  
showStep(2)  
}  
}
```

```
document.getElementById("prev1").onclick = ()=>{  
showStep(1)  
}
```

```
document.getElementById("next2").onclick = ()=>{
```

```
if(skills.length===0){  
alert("Add at least one skill")  
return  
}
```

```
showStep(3)  
}
```

```
document.getElementById("prev2").onclick = ()=>{  
showStep(2)  
}
```

```
function updateProgress(){
```



```
let progress = 0
```

```
if(nameInput.value) progress += 25  
if(emailInput.value) progress += 25  
if(ageInput.value) progress += 25  
if(skills.length>0) progress += 25
```

```
progressBar.value = progress  
percent.innerText = progress + "%"  
}
```

```
document.querySelectorAll("input").forEach(input=>{  
input.addEventListener("input", saveData)  
})
```

```
window.onload = loadData
```

```
*****
```

Problem:2

Html file

```
<!DOCTYPE html>  
<html lang="en">  
<head>  
<meta charset="UTF-8">  
<title>Product Catalog</title>  
<link rel="stylesheet" href="style.css">  
</head>  
<body>  
  
<h2>Product Catalog</h2>  
  
<select id="categoryFilter">  
  <option value="all">All</option>  
  <option value="electronics">Electronics</option>  
  <option value="fashion">Fashion</option>  
</select>  
  
<div class="product-grid">  
  
  <div class="product electronics">  
    <h3>Laptop</h3>  
    <p>Electronics</p>  
  </div>  
  
  <div class="product fashion">  
    <h3>Shoes</h3>  
    <p>Fashion</p>  
  </div>  
  
  <div class="product fashion">  
    <h3>Watch</h3>  
    <p>Fashion</p>  
  </div>  
  
  <div class="product electronics">  
    <h3>Phone</h3>  
    <p>Electronics</p>  
  </div>
```

```
<div class="product electronics">
  <h3>Camera</h3>
  <p>Electronics</p>
</div>
```

```
</div>
```

```
<script src="script.js"></script>
```

```
</body>
```

```
</html>
```

```
Css file:
```

```
body{
  font-family: Arial;
  padding:20px;
}
```

```
h2{
  margin-bottom:20px;
}
```

```
select{
  padding:8px;
  margin-bottom:20px;
}
```

```
/* grid */
```

```
.product-grid{
  display:grid;
  grid-template-columns: repeat(4,1fr);
  gap:20px;
}
```

```
/* product */
```

```
.product{
  background:#f4f4f4;
  padding:20px;
  border-radius:8px;
  text-align:center;
  transition: opacity 0.4s ease, transform 0.4s ease;
}
```

```
/* hidden state */
```

```
.product.hide{
  opacity:0;
  transform:scale(0.9);
  pointer-events:none;
}
```

```
/* tablet */
```

```
@media (max-width:900px){
  .product-grid{
    grid-template-columns: repeat(2,1fr);
  }
}
```

```

}

/* mobile */

@media (max-width:500px){
  .product-grid{
    grid-template-columns: 1fr;
  }
}
Js file:
const filter = document.getElementById("categoryFilter");
const products = document.querySelectorAll(".product");

```

```

filter.addEventListener("change", function(){

  const selected = this.value;

  products.forEach(product => {

    if(selected === "all"){
      product.classList.remove("hide");
    }
    else if(product.classList.contains(selected)){
      product.classList.remove("hide");
    }
    else{
      product.classList.add("hide");
    }

  });

});
*****

```

Problem3:

Html file:

```

<!DOCTYPE html>
<html>
<head>
<title>Weather Preference</title>
</head>
<body>

<h2>Location-Based Weather Preference</h2>

<button id="getLocation">Get My Location</button>

<p id="locationDisplay"></p>
<p id="errorMsg" style="color:red;"></p>

<h3>Select Temperature Unit</h3>

<label>
<input type="radio" name="unit" value="celsius"> Celsius
</label>

<label>
<input type="radio" name="unit" value="fahrenheit"> Fahrenheit
</label>

```

<p id="unitDisplay"></p>

<script src="script.js"></script>

</body>

</html>

Css file:

```
body{
  font-family: Arial;
  padding:20px;
}
```

```
h2{
  margin-bottom:20px;
}
```

```
select{
  padding:8px;
  margin-bottom:20px;
}
```

/* grid */

```
.product-grid{
  display:grid;
  grid-template-columns: repeat(4,1fr);
  gap:20px;
}
```

/* product */

```
.product{
  background:#f4f4f4;
  padding:20px;
  border-radius:8px;
  text-align:center;
  transition: opacity 0.4s ease, transform 0.4s ease;
}
```

/* hidden state */

```
.product.hide{
  opacity:0;
  transform:scale(0.9);
  pointer-events:none;
}
```

/* tablet */

```
@media (max-width:900px){
  .product-grid{
    grid-template-columns: repeat(2,1fr);
  }
}
```

/* mobile */

```
@media (max-width:500px){
```

```

.product-grid{
  grid-template-columns: 1fr;
}
}
Js file:
const button = document.getElementById("getLocation");
const display = document.getElementById("locationDisplay");
const errorMsg = document.getElementById("errorMsg");
const units = document.querySelectorAll('input[name="unit"]');

// GET LOCATION BUTTON
button.addEventListener("click", () => {

  if(navigator.geolocation){

    navigator.geolocation.getCurrentPosition(showPosition, showError);

  }else{
    errorMsg.textContent = "Geolocation not supported by this browser.";
  }

});

// SHOW COORDINATES
function showPosition(position){

  const lat = position.coords.latitude;
  const lon = position.coords.longitude;

  display.innerHTML = `Latitude: ${lat} <br> Longitude: ${lon}`;
}

// HANDLE ERRORS
function showError(error){

  switch(error.code){

    case error.PERMISSION_DENIED:
      errorMsg.textContent = "Permission denied by user.";
      break;

    case error.POSITION_UNAVAILABLE:
      errorMsg.textContent = "Location information unavailable.";
      break;

    case error.TIMEOUT:
      errorMsg.textContent = "Location request timed out.";
      break;

    default:
      errorMsg.textContent = "An unknown error occurred.";
  }

}

// SAVE PREFERENCE
units.forEach(unit => {

  unit.addEventListener("change", () => {

```

```
    sessionStorage.setItem("tempUnit", unit.value);

  });

});
// RESTORE PREFERENCE

window.addEventListener("load", () => {

  const savedUnit = sessionStorage.getItem("tempUnit");
  const display = document.getElementById("unitDisplay");

  if(savedUnit){

    document.querySelector(`input[value="${savedUnit}"]`).checked = true;
    display.textContent = "Preferred Unit: " + savedUnit;

  }

});
```